

Hinglish Sentiment Analyzer

A NLP Pipeline for Code-Mixed Hindi-English Text

Ayush Kundu
IIT Kanpur

Abstract

Hinglish-a code-mixed blend of Hindi and English-is the dominant mode of informal digital communication across India, appearing in social media posts, product reviews, and messaging platforms. Standard NLP tools trained on monolingual corpora fail to handle its inconsistent spelling, free-mixing grammar, and transliteration variability. This project presents an end-to-end sentiment analysis pipeline for Hinglish text using traditional NLP techniques: custom text preprocessing with spelling normalization, Word2Vec embeddings trained from scratch on a 21,369-sentence Hinglish corpus, average word-vector sentence representations, and supervised classification using Support Vector Machines and XGBoost. The best model (XGBoost) achieves a macro F1-score of **0.645** on a held-out test set, demonstrating that carefully engineered classical methods can produce competitive results on code-mixed text without requiring large pretrained transformers.

Contents

1	Introduction	1
2	Why is This Problem Challenging?	1
3	Dataset	1
4	Proposed Pipeline	2
5	Methodology	3
5.1	Text Preprocessing	3
5.2	Word2Vec Embeddings	3
5.3	Sentence Representation (Avg-Word2Vec)	4
5.4	Classification	4
6	Results	4
6.1	Overall Performance	4
6.2	Per-Class Performance (XGBoost)	5
6.3	Confusion Matrices	5
6.4	Sample Predictions	5
7	Tech Stack	6
8	Limitations and Future Work	6
9	Conclusion	7

1 Introduction

Sentiment analysis—the task of identifying whether a piece of text expresses a positive, negative, or neutral opinion—is a well-studied problem in Natural Language Processing. However, most existing tools and pretrained models are designed for clean, monolingual text (primarily English). In multilingual countries like India, everyday digital communication rarely follows this assumption. Instead, users write in *Hinglish*: a natural, unstructured blend of Hindi words written in the Roman script mixed freely with English within the same sentence.

Hinglish appears heavily across Twitter/X posts, YouTube and Instagram comments, product reviews on Amazon and Flipkart, and WhatsApp conversations. Table 1 shows representative examples.

Hinglish Text	Sentiment
Yaar yeh movie bahut boring thi!	Negative
Bhai kal ka match toh ekdum mast tha!	Positive
Theek hi tha, kuch khaas nahi	Neutral
Yeh product bilkul bakwaas hai, waste of money	Negative
Bohot accha experience tha, definitely recommend karunga	Positive

Table 1: Sample Hinglish sentences and their sentiment labels.

The goal of this project is: given a Hinglish text input, predict its sentiment polarity as **Positive**, **Negative**, or **Neutral**, along with a confidence score.

2 Why is This Problem Challenging?

Standard NLP tools fail on Hinglish for several interrelated reasons:

1. **No fixed spelling.** The same Hindi word can be romanized in many ways by different users. For example, the word for “no” may appear as **nahi**, **nhi**, **nahin**, or **nai**—all within the same dataset.
2. **No grammar rules.** Hindi and English mix freely within a single sentence with no predictable structure or code-switching pattern.
3. **Pretrained models are ineffective.** Models like BERT and standard Word2Vec embeddings (e.g., Google News vectors) were trained on clean monolingual corpora and have no representation of code-mixed vocabulary such as **bakwaas**, **mast**, or **shandaar**.
4. **Sarcasm and slang.** A sentence like “Haan bilkul, bahut acha kiya” may be deeply sarcastic depending on context, which surface-level models cannot detect.
5. **Transliteration inconsistency.** The same Hindi word can be romanized in dozens of ways across users and platforms.

These challenges mean one cannot simply plug in existing tools—custom preprocessing and embeddings must be built from scratch.

3 Dataset

The training corpus was assembled by merging multiple publicly available Hinglish sentiment datasets:

Source	Samples	Description
SemEval-2020 Task 9 (Train)	14,594	Hinglish tweets
SemEval-2020 Task 9 (Validation)	3,000	Hinglish tweets
SemEval-2020 Task 9 (Test + Labels)	2,997	Hinglish tweets
Additional Hinglish corpus	2,766	Mixed-domain sentences
Total (after deduplication)	21,369	

Table 2: Dataset composition.

After merging and deduplicating (using lowercased text matching), the final dataset contains 21,369 unique labeled sentences. The class distribution is shown in Figure 1.

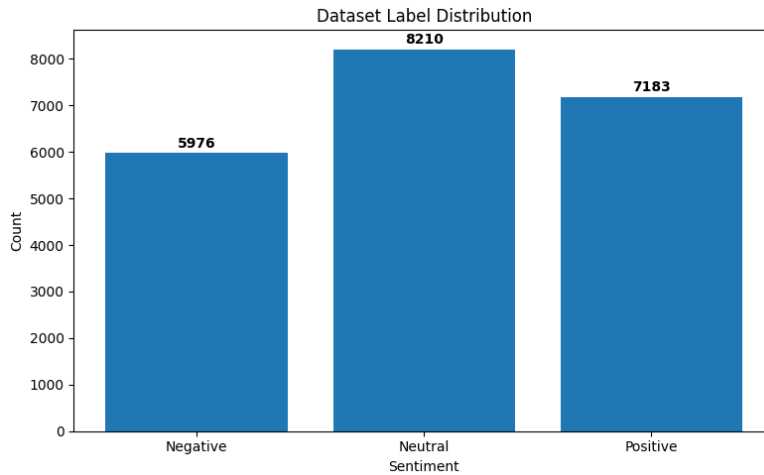


Figure 1: Class distribution across the merged dataset. The dataset is moderately imbalanced, with Neutral being the majority class.

4 Proposed Pipeline

The end-to-end architecture follows six stages, as outlined in Table 3 and illustrated in Figure 2.

Step	Stage	Description
1	Raw Text Input	Hinglish tweets, reviews, or comments
2	Preprocessing	Cleaning, normalization, tokenization
3	Word2Vec Training	Train skip-gram embeddings on Hinglish corpus
4	Avg-Word2Vec	Compute sentence-level vector representation
5	Classification	SVM or XGBoost trained on sentence vectors
6	Sentiment Output	Positive / Negative / Neutral with confidence

Table 3: End-to-end project pipeline.

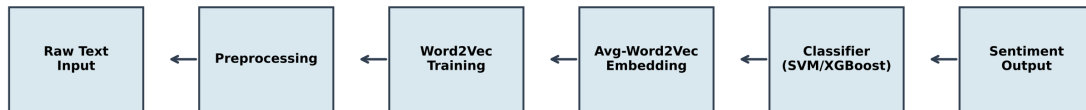


Figure 2: Visual representation of the sentiment analysis pipeline.

5 Methodology

5.1 Text Preprocessing

Raw Hinglish text undergoes the following preprocessing steps:

1. **Lowercasing** all text to ensure case-insensitive matching.
2. **URL and mention removal** using regex patterns (`http\S+`, `@\w+`).
3. **Hashtag normalization**: converting `#trending` to `trending`.
4. **Punctuation and number removal** to reduce noise.
5. **Spelling normalization** using a hand-crafted dictionary that maps common Hinglish spelling variants to canonical forms. Examples are shown in Table 4.
6. **Stopword removal** using a combined Hindi-English stopwords list (e.g., `ka`, `ki`, `ke`, `the`, `is`, `are`).

Variants	Canonical Form	Meaning
nhi, nahin, nai	nahi	no
bohot, boht, bhot	bahut	very
accha, achha, achchha	acha	good
bakwaas, bakwass	bakwas	nonsense
bekaar	bekar	useless
shandaar	shandar	magnificent

Table 4: Examples from the spelling normalization dictionary.

5.2 Word2Vec Embeddings

Since pretrained English embeddings do not cover Hinglish vocabulary, we train a **Skip-Gram Word2Vec** model from scratch on the full corpus (including stopwords, to preserve contextual co-occurrence information). The hyperparameters are:

Parameter	Value
Architecture	Skip-Gram (sg=1)
Vector dimension	100
Context window	5
Minimum word count	3
Negative sampling	10
Training epochs	20

Table 5: Word2Vec training hyperparameters.

The trained model contains **11,430 vocabulary words**, each represented as a 100-dimensional dense vector. Qualitative inspection confirms that semantically related words cluster together—for example, `modi` is most similar to `narendra`, `pm`, and `ji`; while `hindu` is most similar to `muslim`, `musalman`, and `kattar`.

5.3 Sentence Representation (Avg-Word2Vec)

Each sentence is represented as the element-wise mean of all its token embeddings:

$$\mathbf{v}_{\text{sentence}} = \frac{1}{N} \sum_{i=1}^N \mathbf{w}_i \quad (1)$$

where N is the number of tokens in the sentence and $\mathbf{w}_i \in \mathbb{R}^{100}$ is the Word2Vec embedding of the i -th token. Tokens not found in the vocabulary are ignored. If all tokens are out-of-vocabulary, the sentence is represented as the zero vector.

This produces a fixed-size 100-dimensional feature vector for each sentence, regardless of sentence length, which can be directly fed into any standard classifier.

5.4 Classification

Two classifiers are trained on the sentence vectors:

1. **Linear SVM** (LinearSVC with $C = 1.0$, wrapped in `CalibratedClassifierCV` with 3-fold cross-validation to obtain probability estimates).
2. **XGBoost** (200 estimators, max depth 6, learning rate 0.1, multi-class log-loss objective).

The dataset is split into 80% training (17,095 samples) and 20% testing (4,274 samples) using stratified sampling to preserve class proportions. The best-performing model is retrained on the full dataset for deployment.

6 Results

6.1 Overall Performance

Table 6 compares the two classifiers on the held-out test set.

Model	Macro F1	Accuracy
Linear SVM	0.601	59.7%
XGBoost	0.645	64.2%

Table 6: Model comparison on the test set. XGBoost outperforms SVM on both metrics.

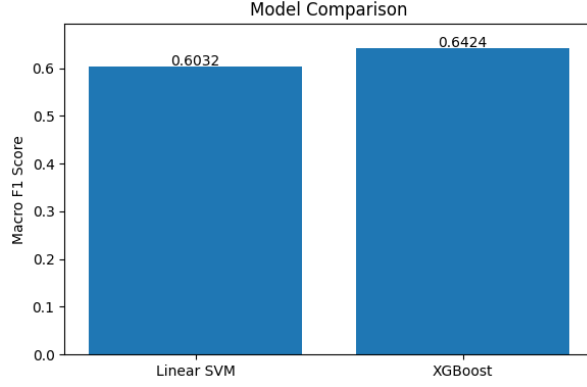


Figure 3: Macro F1 and accuracy comparison between SVM and XGBoost.

6.2 Per-Class Performance (XGBoost)

Table 7 shows the detailed classification report for the best model.

Class	Precision	Recall	F1-Score
Negative	0.67	0.63	0.65
Neutral	0.59	0.61	0.60
Positive	0.68	0.69	0.69

Table 7: Per-class precision, recall, and F1-score for XGBoost.

The **Positive** class achieves the highest F1-score (0.69), while **Neutral** is the hardest to classify ($F1 = 0.60$). This is expected: neutral sentences often lack strong sentiment-bearing words and overlap lexically with both positive and negative classes.

6.3 Confusion Matrices

Figure 4 shows the confusion matrices for both models. The most common error pattern is confusion between Neutral and the other two classes, consistent with the inherent ambiguity of neutral sentiment.

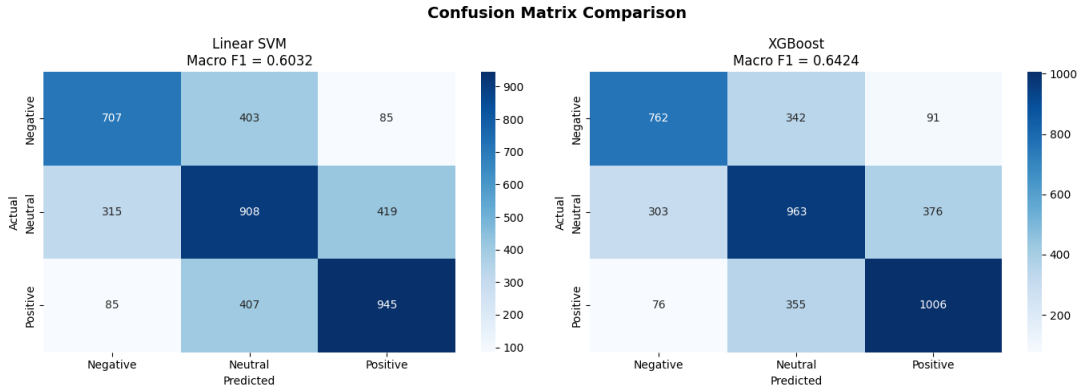


Figure 4: Confusion matrices for SVM (left) and XGBoost (right).

6.4 Sample Predictions

Table 8 shows predictions from the deployed XGBoost model on unseen sentences.

Input	Predicted	Confidence
Yaar yeh movie bahut boring thi!	Negative	71.4%
Bhai kal ka match toh ekdum mast tha!	Positive	62.8%
Theek hi tha, kuch khaas nahi	Neutral	65.2%
Yeh product bilkul bakwaas hai, waste of money	Negative	84.3%
Bohot accha experience tha, recommend karunga	Positive	72.1%

Table 8: Sample predictions from the final model.

7 Tech Stack

Component	Tool / Library
Language	Python 3
Embeddings	Gensim (Word2Vec)
Classifiers	scikit-learn (LinearSVC), XGBoost
Data handling	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Evaluation	sklearn.metrics (F1, confusion matrix)
Development	Google Colab (Jupyter Notebook)

Table 9: Technology stack used in the project.

8 Limitations and Future Work

While the pipeline achieves reasonable performance using traditional methods, several limitations remain:

1. **Loss of word order.** The Avg-Word2Vec representation treats each sentence as a bag of words, discarding sequential information. This makes the model unable to distinguish “not good” from “good” if negation handling is imperfect.
2. **Sarcasm detection.** Sarcastic sentences (e.g., “Haan bilkul, bahut acha kiya”) are misclassified because the model relies on surface-level lexical features.
3. **Out-of-vocabulary words.** Extremely rare spellings or neologisms not seen during Word2Vec training are ignored during inference.
4. **Neutral class ambiguity.** Neutral sentences are inherently harder to define and classify, dragging down the overall macro F1.

Potential improvements for future work include:

- Using TF-IDF weighted averaging instead of simple mean pooling for sentence vectors.
- Incorporating subword embeddings (FastText) to handle spelling variants more robustly.
- Exploring lightweight sequence models (BiLSTM) to capture word order.
- Fine-tuning multilingual transformer models (e.g., XLM-RoBERTa) on Hinglish data.
- Expanding the normalization dictionary using automated methods.

9 Conclusion

This project demonstrates that a carefully engineered traditional NLP pipeline—combining custom preprocessing, corpus-specific Word2Vec embeddings, and gradient-boosted classification—can achieve meaningful sentiment analysis on code-mixed Hinglish text. The final XGBoost model achieves a macro F1-score of 0.645 on a 21,369-sentence corpus, with the Positive class reaching 0.69 F1 and Neutral being the primary source of errors. The approach is lightweight, interpretable, and deployable without GPU resources, making it suitable for practical applications and academic demonstrations alike.